ỦY BAN NHÂN DÂN TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



Họ và tên sinh viên: Đào Duy Hưng

Mã số sinh viên: 3122410158

EX01 TÌM KIẾM TRONG AI

Môn học : **Trí tuệ nhân tạo nâng cao**

Giảng viên : **ThS. Đỗ Như Tài**

TP. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1: Tìm kiếm AI..... 2
Bài tập 1: Các thuật toán tìm kiếm..... 2
1.1. Breadth-First Search (BFS)..... 2
1.2. Depth-First Search (DFS)..... 4
1.3. Iterative Deepening Search (IDS)..... 7
Bài tập 2: Tháp Hà Nội - Tower of Hanoi..... 10
2.1. Biểu diễn trạng thái..... 10
2.2. Kích thước không gian trạng thái..... 11
2.3. Trạng thái bắt đầu..... 13
2.4. Các hành động hợp lệ..... 13
2.5. Goal Test..... 16
Bài tập 3: Thiết kế và phân tích Heuristic..... 17
3.1. Thiết kế heuristic admissible và consistent..... 18
3.2. Chạy A* với heuristic đã thiết kế..... 24
3.3. Heuristic admissible nhưng không consistent..... 28
3.4. Câu hỏi mở rộng..... 31
Bài tập 4: Treasure Hunting (Tìm kho báu)..... 33
4.1. Depth-First Search..... 33
4.2. Breadth-First Search..... 34
4.3. Uniform-Cost Search..... 34
4.4. Greedy Search..... 34
4.5. A* Search..... 35
Bài tập 5: Câu hỏi đúng/sai..... 36
Câu 1..... 36
Câu 2..... 36
Câu 3..... 38
Câu 4..... 38
Câu 5..... 40
Câu 6..... 41
CHƯƠNG 2: TÌM KIẾM VÀ HEURISTIC TRONG AI..... 43
Câu 1. Tìm kiếm và heuristic..... 43
Câu 2. Các nút tìm kiếm..... 44
Câu 3. Tìm kiếm trên đồ thị..... 45

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm.....47

LỜI MỞ ĐẦU

Trong bối cảnh trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong học tập, nghiên cứu và thực tiễn, việc hiểu rõ các nguyên lý, phương pháp và cách xây dựng hệ thống trí tuệ nhân tạo là nền tảng cần thiết đối với sinh viên theo định hướng công nghệ. Môn học Trí tuệ nhân tạo nâng cao giúp người học tiếp cận các kiến thức và vấn đề cốt lõi của lĩnh vực AI, đồng thời rèn luyện khả năng phân tích, tư duy logic và vận dụng kiến thức vào những bài toán thực tế.

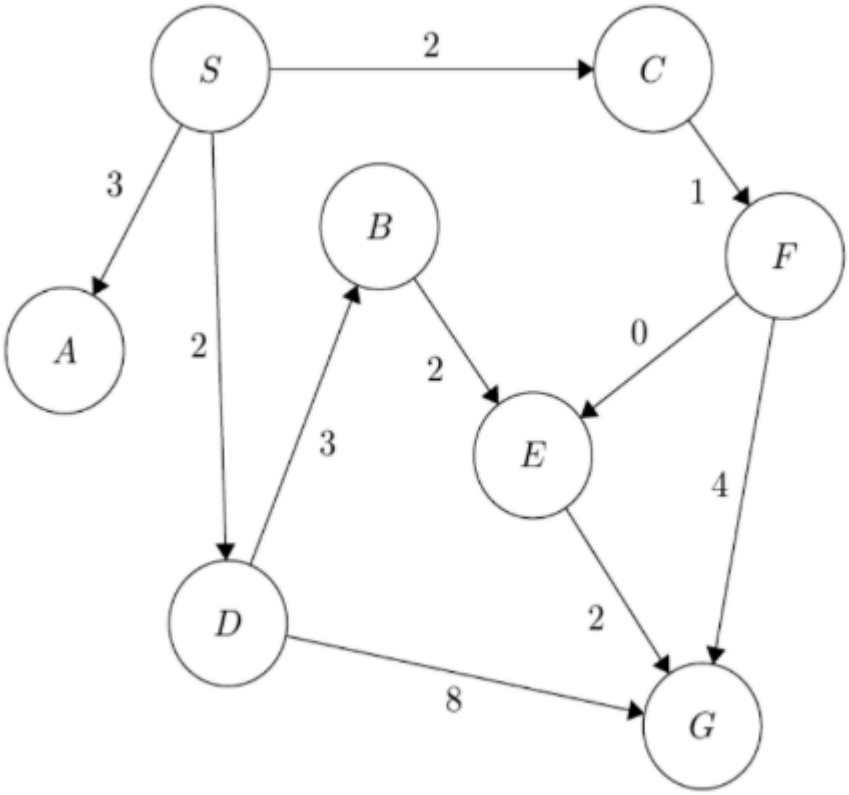
Assignment này được thực hiện nhằm củng cố và hệ thống hóa những kiến thức đã được học thông qua việc nghiên cứu, phân tích và trả lời các câu hỏi theo từng chủ đề. Thay vì chỉ tiếp cận kiến thức ở mức lý thuyết, quá trình thực hiện bài tập giúp tôi hiểu rõ hơn cách các khái niệm AI được hình thành, cách chúng liên kết với nhau và cách có thể áp dụng vào các bài toán cụ thể.

Trong quá trình thực hiện, tôi tập trung vào việc tự tìm hiểu nội dung, đối chiếu với tài liệu môn học, phân tích vấn đề và trình bày kết quả theo hướng rõ ràng, có lập luận. Qua đó, tôi không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện khả năng nghiên cứu tài liệu, giải quyết vấn đề và đánh giá một hệ thống AI dưới nhiều góc độ khác nhau.

Thông qua assignment này và các bài tập tiếp theo, tôi kỳ vọng có thể xây dựng nền tảng kiến thức vững chắc hơn về Trí tuệ nhân tạo, đồng thời từng bước nâng cao khả năng vận dụng kiến thức vào các bài toán thực tế. Đây cũng là cơ sở để tôi tiếp tục phát triển các kỹ năng chuyên môn và nghiên cứu sâu hơn trong lĩnh vực AI trong quá trình học tập và định hướng nghề nghiệp sau này.

CHƯƠNG 1: Tìm kiếm AI

Bài tập 1: Các thuật toán tìm kiếm



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

1.1. Breadth-First Search (BFS)

BFS luôn ưu tiên node có độ sâu nhỏ nhất. Nếu nhiều node cùng độ sâu thì chọn theo alphabet.

Quá trình duyệt

Ban đầu:

$$Frontier = [S]$$

Lấy S ra, ta có ba node ở độ sâu 1:

$$A,\ C,\ D$$

Theo alphabet:

$$A < C < D$$

Sau khi duyệt A, tiếp tục tới C. Từ C sinh ra F.

Sau đó duyệt D, từ D sinh:

$$B,\ G$$

Ở độ sâu 2 hiện có:

$$B,\ F,\ G$$

Theo quy tắc alphabet:

$$B < F < G$$

Vì vậy thứ tự duyệt là: S, A, C, D, B, F, G

Khi gặp G, thuật toán kết thúc.

Đường đi tới G được tìm thấy ở độ sâu nhỏ nhất là: $S\ ->\ D\ ->\ G$

Chi phí:

$$Cost(S, D) = 2$$

$$Cost(D, G) = 8$$

nên:

$$Total\ Cost = 2 + 8 = 10$$

Điền vào bảng 1.1

Nội dung	Kết quả
Explored Set	**S, A, C, D, B, F, G**
Path	**S → D → G**
Total Cost	**10**

1.2. Depth-First Search (DFS)

DFS ưu tiên đi sâu nhất có thể trước khi quay lui. Khi có nhiều nhánh cùng mức ưu tiên, áp dụng quy tắc alphabet.

Quá trình duyệt

Ban đầu:

$$S$$

Các nhánh từ S:

$$A,\ C,\ D$$

Theo alphabet, đi vào A trước:

$$S \to A$$

A không có node con nên quay lui về S.

Tiếp theo chọn:

$$S \to C$$

Từ C chỉ có:

$$C \to F$$

nên:

$$S \to C \to F$$

Từ F có:

$$E,\ G$$

Theo alphabet:

$$E < G$$

nên DFS tiếp tục xuống E:

$$S \rightarrow C \rightarrow F \rightarrow E$$

Từ E:

$$E \rightarrow G$$

và gặp trạng thái đích.

Do đó thứ tự duyệt: S, A, C, F, E, G

Đường đi được trả về: $S \ -> \ C \ -> \ F \ -> \ E \ -> \ G$

Chi phí:

$$S \rightarrow C = 2$$

$$C \rightarrow F = 1$$

$$F \rightarrow E = 0$$

$$E \rightarrow G = 2$$

Do đó:

$$Total\ Cost = 2 + 1 + 0 + 2 = 5$$

Điền vào bảng 1.2

Nội dung	Kết quả
Explored Set	**S, A, C, F, E, G**
Path	**S → C → F → E → G**
Total Cost	**5**

1.3. Iterative Deepening Search (IDS)

IDS thực hiện **Depth-Limited Search** nhiều lần với giới hạn độ sâu tăng dần:

$$0, 1, 2, \ldots$$

Mỗi lần tăng giới hạn, thuật toán bắt đầu tìm lại từ S. Phần này của đề yêu cầu ghi các node được duyệt theo từng giới hạn độ sâu. BT01 - Tim_kiem_AI

Giới hạn độ sâu = 0

Chỉ được xét node gốc: S

Không tìm thấy G.

Giới hạn độ sâu = 1

Bắt đầu lại từ S.

Ở mức 1 có:

$$A,\ C,\ D$$

Theo alphabet:

$$A < C < D$$

Thứ tự duyệt:S, A, C, D

Chưa gặp G.

Giới hạn độ sâu = 2

Bắt đầu lại từ S.

DFS giới hạn độ sâu 2 hoạt động như sau:

$$S$$

Đầu tiên:

$$S \rightarrow A$$

A không có node con.

Quay lại rồi:

$$S \rightarrow C \rightarrow F$$

F đã ở độ sâu 2 nên không mở rộng tiếp.

Quay lại:

$$S \rightarrow D$$

Các node con của D là:

$$B,\ G$$

Theo alphabet:

$$B < G$$

nên xét B trước, sau đó G:

$$S, A, C, F, D, B, G$$

Tại G, Goal Test thành công.

Vì vậy: S, A, C, F, D, B, G

Đường đi được IDS trả về: $S\ ->\ D\ ->\ G$

Chi phí: 2 + 8 = 10

Điền bảng 1.3

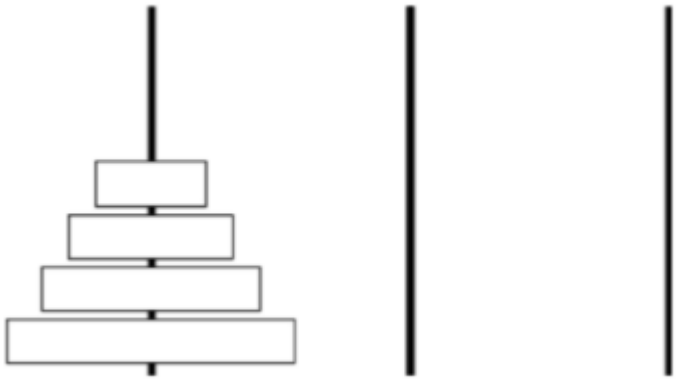
Giới hạn độ sâu	**Các nút được duyệt**

0	**S**
1	**S, A, C, D**
2	**S, A, C, F, D, B, G**
...	**Dừng vì đã tìm thấy G ở giới hạn 2**

Path cuối cùng: $S \ -> D \ -> \ G$

Total Cost: 10

Bài tập 2: Tháp Hà Nội - Tower of Hanoi



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Ta biểu diễn một trạng thái bằng vector:

$$s = (p_1, p_2, ..., p_n)$$

trong đó:

$$p_i \in \{A, B, C\}$$

là cọc hiện đang chứa đĩa i.

Ví dụ với 4 đĩa:

$$s = (A, A, B, C)$$

có nghĩa là:

- đĩa 1 ở cọc A,
- đĩa 2 ở cọc A,
- đĩa 3 ở cọc B,
- đĩa 4 ở cọc C.

Do kích thước các đĩa là khác nhau nên khi biết các đĩa thuộc cọc nào, thứ tự hợp lệ trên mỗi cọc được xác định duy nhất: đĩa lớn luôn nằm dưới đĩa nhỏ. Vì vậy cách biểu diễn này đủ để xác định trạng thái cũng như các hành động có thể thực hiện. Yêu cầu của đề cũng nhấn mạnh rằng biểu diễn trạng thái phải cho biết mỗi đĩa đang ở cọc nào và phải đủ để xác định các hành động hợp lệ.

2.2. Kích thước không gian trạng thái

Mỗi đĩa có thể nằm trên một trong ba cọc:

$$A,\ B,\ C$$

Đối với mỗi đĩa có:

$$3$$

lựa chọn.

Với n đĩa, số cách gán các đĩa vào ba cọc là:

$$3 \times 3 \times \cdots \times 3$$

gồm n thừa số.

Do đó kích thước không gian trạng thái là:

$$|S| = 3^n$$

Lý do mọi cách gán như trên đều tương ứng với một cấu hình hợp lệ là vì trên mỗi cọc, thứ tự các đĩa được xác định tự động theo kích thước: đĩa lớn hơn ở dưới, đĩa nhỏ hơn ở trên.

Trong hình của đề có 4 đĩa, do đó nếu xét trường hợp cụ thể này:

$$n = 4$$

ta có:

$$|S| = 3^4 = 81$$

trạng thái.

2.3. Trạng thái bắt đầu

Theo đề bài, ban đầu toàn bộ n đĩa đều nằm trên cọc thứ nhất A, được xếp từ lớn đến nhỏ.

Theo cách biểu diễn đã chọn:

$$S_0 = (A, A, ..., A)$$

gồm n phần tử A.

Với trường hợp 4 đĩa trong hình:

$$S_0 = (A, A, A, A)$$

Trong đó:

$$p_1 = p_2 = p_3 = p_4 = A$$

2.4. Các hành động hợp lệ

Từ một trạng thái s, một hành động có dạng:

$$Move(i, X, Y)$$

nghĩa là di chuyển đĩa i từ cọc X sang cọc Y, trong đó:

$$X \neq Y$$

Một hành động chỉ hợp lệ khi thỏa mãn hai điều kiện chính.

Điều kiện 1: Đĩa cần di chuyển phải nằm trên cùng của cọc nguồn

Không được có một đĩa nhỏ hơn nằm phía trên đĩa i.

Nếu đánh số đĩa theo thứ tự:

$$1 < 2 < \cdots < n$$

với đĩa 1 nhỏ nhất, thì không được tồn tại đĩa j sao cho:

$$j < i$$

và:

$$p_j = X$$

Nói cách khác, đĩa i phải là đĩa nhỏ nhất hiện có trên cọc nguồn X.

Điều kiện 2: Không được đặt đĩa lớn lên đĩa nhỏ

Cọc đích Y phải:

- đang trống, hoặc
- có đĩa trên cùng lớn hơn đĩa i.

Nếu đĩa trên cùng của cọc Y là đĩa j, thì phải thỏa:

$$i < j$$

vì đĩa i nhỏ hơn đĩa j.

Sau khi thực hiện hành động:

$$Move(i, X, Y)$$

trạng thái mới được tạo bằng cách đổi:

$$p_i = X$$

thành:

$$p_i = Y$$

các đĩa còn lại giữ nguyên.

Ví dụ

Giả sử:

$$s = (A, A, A, A)$$

chỉ đĩa 1 nằm trên cùng của cọc A.

Các hành động hợp lệ là:

$$Move(1, A, B)$$

hoặc:

$$Move(1, A, C)$$

Không thể di chuyển đĩa 2, 3 hoặc 4 vì các đĩa này đang bị đĩa nhỏ hơn nằm phía trên.

2.5. Goal Test

Mục tiêu của bài toán là chuyển toàn bộ n đĩa sang cọc thứ ba, tức cọc C.

Do đó trạng thái đích là:

$$S_G = (C, C, \ldots, C)$$

Hàm:

$$goalTest(state)$$

trả về `True` khi:

$$p_i = C, \forall i \in \{1, 2, \ldots, n\}$$

hay tương đương:

$$state = (C, C, \ldots, C)$$

Ngược lại, nếu còn ít nhất một đĩa không nằm trên cọc C, hàm trả về `False`.

Với 4 đĩa:

$$S_G = (C, C, C, C)$$

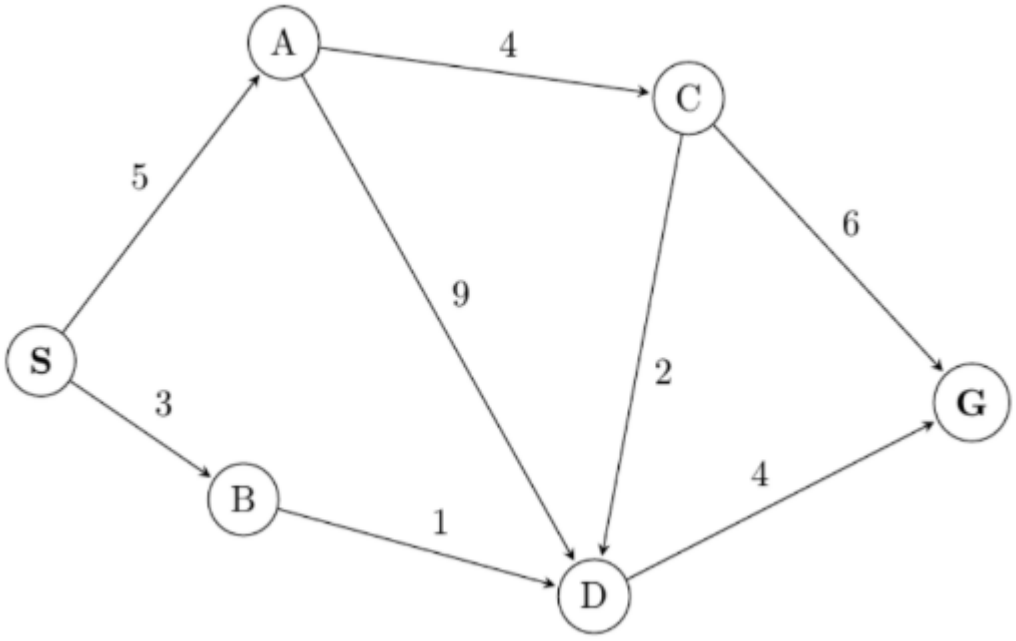
Do đó:

$$goalTest(s) = True$$

khi:

$$s = (C, C, C, C)$$

Bài tập 3: Thiết kế và phân tích Heuristic



Hình 3. Đồ thị dùng để thiết kế heuristic

3.1. Thiết kế heuristic admissible và consistent

Để thiết kế một heuristic chắc chắn vừa admissible vừa consistent, ta có thể chọn:

$$h(n) = h^*(n)$$

trong đó $h^*(n)$ là chi phí thực sự nhỏ nhất từ node n đến G.

Tính chi phí tối ưu từ từng node đến G

Từ G:

$$h^*(G) = 0$$

Từ D:

$$D \to G = 4$$

nên:

$$h^*(D) = 4$$

Từ C, có hai đường:

$$C \to G = 6$$

và:

$$C \to D \to G = 2 + 4 = 6$$

nên:

$$h^*(C) = 6$$

Từ B:

$$B \to D \to G$$

$$= 1 + 4 = 5$$

nên:

$$h^*(B) = 5$$

Từ A:

$$A \to C \to G = 4 + 6 = 10$$

hoặc:

$$A \to D \to G = 9 + 4 = 13$$

Do đó:

$$h^*(A) = 10$$

Cuối cùng, từ S:

Qua A:

$$S \to A \to C \to G$$

$$= 5 + 4 + 6 = 15$$

Qua B:

$$S \to B \to D \to G$$

$$= 3 + 1 + 4 = 8$$

nên:

$$h^*(S) = 8$$

Bảng heuristic

Node	**h(n)**
S	8

A	10
B	5
C	6
D	4
G	0

Bộ heuristic được chọn là:

$$h(S) = 8, h(A) = 10, h(B) = 5$$

$$h(C) = 6, h(D) = 4, h(G) = 0$$

Kiểm tra admissible

Điều kiện admissible:

$$h(n) \leq h^*(n)$$

Ở đây ta chọn chính xác:

$$h(n) = h^*(n)$$

với mọi node, do đó heuristic chắc chắn admissible.

Kiểm tra consistent

Điều kiện consistency đối với mỗi cạnh $n \to n'$:

$$h(n) \leq c(n, n') + h(n')$$

Ta kiểm tra:

$$S \to A:$$

$$8 \leq 5 + 10 = 15$$

Đúng.

$$S \to B:$$

$$8 \leq 3 + 5 = 8$$

Đúng.

$$A \to C:$$

$$10 \leq 4 + 6 = 10$$

Đúng.

$$A \rightarrow D:$$

$$10 \leq 9 + 4 = 13$$

Đúng.

$$B \rightarrow D:$$

$$5 \leq 1 + 4 = 5$$

Đúng.

$$C \rightarrow D:$$

$$6 \leq 2 + 4 = 6$$

Đúng.

$$C \rightarrow G:$$

$$6 \leq 6 + 0 = 6$$

Đúng.

$$D \rightarrow G:$$

$$4 \leq 4 + 0 = 4$$

Đúng.

Như vậy heuristic trên vừa admissible vừa consistent.

3.2. Chạy A* với heuristic đã thiết kế

A* lựa chọn node có:

$$f(n) = g(n) + h(n)$$

nhỏ nhất. Phần này của PDF yêu cầu xác định Explored Set, Path và Total Cost. BT01 - Tim_kiem_AI

Bước 1: Bắt đầu tại S

$$g(S) = 0$$

$$h(S) = 8$$

$$f(S) = 0 + 8 = 8$$

Ta mở rộng S.

Hai node con là A và B.

Đối với A:

$$g(A) = 5$$

$$h(A) = 10$$

$$f(A) = 5 + 10 = 15$$

Đối với B:

$$g(B) = 3$$

$$h(B) = 5$$

$$f(B) = 3 + 5 = 8$$

Frontier:

Node	**g(n)**	**h(n)**	**f(n)**
B	3	5	8
A	5	10	15

Node có f nhỏ nhất là B.

Bước 2: Mở rộng B

Từ B:

$$B \rightarrow D = 1$$

Do đó:

$$g(D) = g(B) + 1$$

$$g(D) = 3 + 1 = 4$$

$$h(D) = 4$$

$$f(D) = 4 + 4 = 8$$

Frontier:

Node	g(n)	h(n)	f(n)
D	4	4	8
A	5	10	15

Chọn D.

Bước 3: Mở rộng D

Từ D:

$$D \rightarrow G = 4$$

nên:

$$g(G) = g(D) + 4$$

$$g(G) = 4 + 4 = 8$$

Vì:

$$h(G) = 0$$

nên:

$$f(G) = 8 + 0 = 8$$

Frontier:

Node	g(n)	h(n)	f(n)
G	8	0	8

A 5 10 15

A* chọn G.

G là trạng thái đích nên thuật toán dừng.

Explored Set:

$$S,\ B,\ D,\ G$$

Path:

$$S \rightarrow B \rightarrow D \rightarrow G$$

Total Cost:

$$3 + 1 + 4 = 8$$

Do đó:

$$Total\ Cost = 8$$

3.3. Heuristic admissible nhưng không consistent

Đề yêu cầu tạo một bộ $h(n)$ vẫn admissible, nhưng vi phạm ít nhất một điều kiện consistency. BT01 - Tim_kiem_AI

Ta chọn:

Node	h(n)	h*(n)
S	8	8
A	10	10
B	0	5
C	6	6
D	4	4
G	0	0

Tức là:

$$h(S) = 8$$

$$h(A) = 10$$

$$h(B) = 0$$

$$h(C) = 6$$

$$h(D) = 4$$

$$h(G) = 0$$

Kiểm tra admissible

Với tất cả các node:

$$h(n) \leq h^{*}(n)$$

Ví dụ:

$$h(S) = 8 \leq 8$$

$$h(B) = 0 \leq 5$$

$$h(D) = 4 \leq 4$$

Do đó heuristic vẫn admissible.

Nhưng không consistent

Xét cạnh:

$$S \rightarrow B$$

có cost:

$$c(S, B) = 3$$

Điều kiện consistency yêu cầu:

$$h(S) \leq c(S, B) + h(B)$$

Thay số:

$$8 \leq 3 + 0$$

$$8 \leq 3$$

Điều này sai.

Do đó cạnh:

$$S \rightarrow B$$

vi phạm consistency.

Vì vậy bộ heuristic trên là:

$$Admissible nhưng không Consistent$$

Tóm lại:

Có thể chọn:

$$h(S) = 8,\ h(A) = 10,\ h(B) = 0,\ h(C) = 6,\ h(D) = 4,\ h(G) = 0$$

Các giá trị trên đều thỏa:

$$h(n) \leq h^{*}(n)$$

nên heuristic admissible. Tuy nhiên với cạnh $S \to B$:

$$h(S) = 8 > c(S, B) + h(B) = 3 + 0 = 3$$

nên heuristic không consistent.

3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử $h(G) = 0$.

Một heuristic consistent thỏa mãn với mọi cạnh:

$$h(n) \leq c(n, n') + h(n')$$

Xét một đường đi tối ưu từ n đến G:

$$n = n_0 \to n_1 \to n_2 \to \cdots \to n_k = G$$

Theo consistency:

$$h(n_0) \leq c(n_0, n_1) + h(n_1)$$

Tiếp tục:

$$h(n_1) \leq c(n_1, n_2) + h(n_2)$$

nên:

$$h(n_0) \leq c(n_0, n_1) + c(n_1, n_2) + h(n_2)$$

Áp dụng liên tiếp đến G:

$$h(n) \leq c(n_0, n_1) + c(n_1, n_2) + \cdots + c(n_{k-1}, G) + h(G)$$

Vì:

$$h(G) = 0$$

nên:

$$h(n) \leq c(n_0, n_1) + c(n_1, n_2) + \cdots + c(n_{k-1}, G)$$

Tổng ở vế phải chính là chi phí tối ưu thực tế từ n đến G:

$$h^*(n)$$

Do đó:

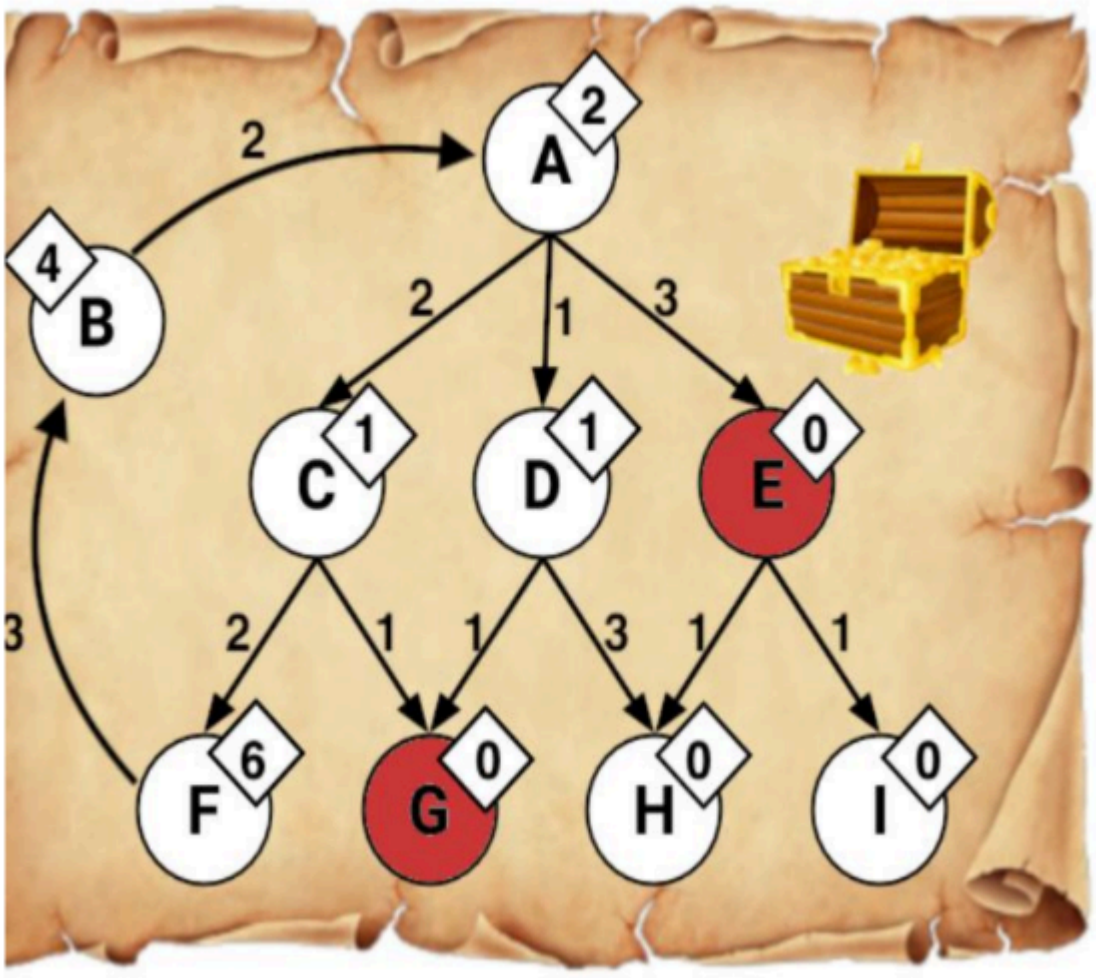
$$h(n) \leq h^*(n)$$

Đây chính là điều kiện của heuristic admissible.

Vì vậy:

$$Consistent \Rightarrow Admissible$$

Bài tập 4: Treasure Hunting (Tìm kho báu)



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

4.1. Depth-First Search

Explored Set:

$$A,\ C,\ F,\ B,\ G$$

Path returned:

$$A \rightarrow C \rightarrow G$$

4.2. Breadth-First Search

Explored Set:

$$A,\ C,\ D,\ E$$

Path returned:

$$A \rightarrow E$$

4.3. Uniform-Cost Search

Explored Set:

$$A,\ D,\ C,\ G$$

Path returned:

$$A \rightarrow D \rightarrow G$$

Cost:

$$2$$

4.4. Greedy Search

Explored Set:

$$A,\ E$$

Path returned:

$$A \rightarrow E$$

Cost:

$$3$$

4.5. A* Search

Explored Set:

$$A,\ D,\ G$$

Path returned:

$$A \rightarrow D \rightarrow G$$

Cost:

$$2$$

Bảng tổng hợp so sánh

Đúng theo bảng cuối của PDF, các tiêu chí là DFS theo độ sâu, BFS theo độ sâu nhỏ nhất, UCS theo $g(n)$, Greedy theo $h(n)$, và A* theo $g(n) + h(n)$. BT01 - Tim_kiem_AI

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	$A \rightarrow C \rightarrow G$	**3**
BFS	Độ sâu nhỏ nhất	$A \rightarrow E$	**3**
UCS	$g(n)$	$A \rightarrow D \rightarrow G$	**2**
Greedy	$h(n)$	$A \rightarrow E$	**3**

A*	$g(n) + h(n)$	$A \to D \to G$	**2**

Bài tập 5: Câu hỏi đúng/sai

Câu 1

Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai: Sai

Giải thích:
DFS không có quan hệ đảm bảo rằng luôn mở rộng nhiều node hơn A*. Trong một số bài toán, DFS có thể tình cờ đi đúng nhánh chứa goal ngay từ đầu và chỉ cần mở rộng rất ít node, trong khi A* vẫn có thể phải mở rộng nhiều node có giá trị $f(n)$ nhỏ hơn trước khi tới goal.

Vì vậy không thể khẳng định:

$$N_{DFS} \geq N_{A^*}$$

trong mọi trường hợp.

Câu 2

Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Đúng / Sai: Sai

Giải thích:
Manhattan distance được tính bởi:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

Nhưng quân xe có thể đi nhiều ô trên cùng một hàng hoặc cột chỉ trong một nước đi.

Ví dụ, từ ô:

$$A1 \rightarrow A8$$

Manhattan distance là:

$$|1 - 1| + |1 - 8| = 7$$

nhưng quân xe chỉ cần:

$$1$$

nước đi.

Do đó:

$$h(n) = 7 > h^*(n) = 1$$

Heuristic đã overestimate chi phí thực tế nên không admissible.

Câu 3

Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích:
Trong bài toán Pacman thông thường, Pacman di chuyển trên lưới theo bốn hướng:

$$Up,\ Down,\ Left,\ Right$$

với mỗi bước có chi phí 1.

Euclidean distance:

$$h(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}$$

là khoảng cách đường thẳng giữa vị trí hiện tại và goal. Đường đi thực tế trên lưới, đặc biệt khi có tường, không thể ngắn hơn khoảng cách đường thẳng.

Do đó:

$$h(n) \leq h^*(n)$$

nên Euclidean distance là một heuristic admissible.

Câu 4

Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích:
Giả sử hai heuristic h_1 và h_2 đều admissible:

$$h_1(n) \leq h^*(n)$$

$$h_2(n) \leq h^*(n)$$

Điều này không có nghĩa là:

$$h_1(n) + h_2(n) \leq h^*(n)$$

Ví dụ:

$$h^*(n) = 10$$

chọn:

$$h_1(n) = 6$$

$$h_2(n) = 6$$

Cả hai đều admissible vì:

$$6 \leq 10$$

nhưng:

$$h_1(n) + h_2(n) = 12$$

và:

$$12 > 10$$

nên tổng của chúng không admissible.

Câu 5

Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích:
Admissible chỉ yêu cầu:

$$h(n) \leq h^*(n)$$

Trong khi consistent yêu cầu điều kiện mạnh hơn, với mọi cạnh $n \to n'$:

$$h(n) \leq c(n, n') + h(n')$$

Một heuristic có thể không đánh giá vượt quá chi phí tối ưu nên admissible, nhưng vẫn vi phạm bất đẳng thức consistency trên một cạnh nào đó.

Ví dụ:

$$S \to A = 1$$

$$A \to G = 2$$

Chi phí tối ưu:

$$h^*(S) = 3, h^*(A) = 2$$

chọn:

$$h(S) = 3, h(A) = 1, h(G) = 0$$

Ta có:

$$h(S) = 3 \leq 3$$

và:

$$h(A) = 1 \leq 2$$

nên heuristic admissible.

Nhưng trên cạnh $S \to A$:

$$h(S) \leq c(S, A) + h(A)$$

sẽ yêu cầu:

$$3 \leq 1 + 1$$

hay:

$$3 \leq 2$$

điều này sai.

Vậy heuristic admissible nhưng không consistent.

Câu 6

A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích:
Đối với A* Tree Search, heuristic admissible đủ để đảm bảo tính tối ưu trong các điều kiện chuẩn.

Tuy nhiên với A* Graph Search, nếu một node đã được đưa vào explored set và không được mở lại, chỉ admissible thôi là chưa đủ. Heuristic có thể admissible nhưng không consistent, làm A* đóng một node theo một đường có cost cao trước khi tìm được đường tốt hơn tới node đó.

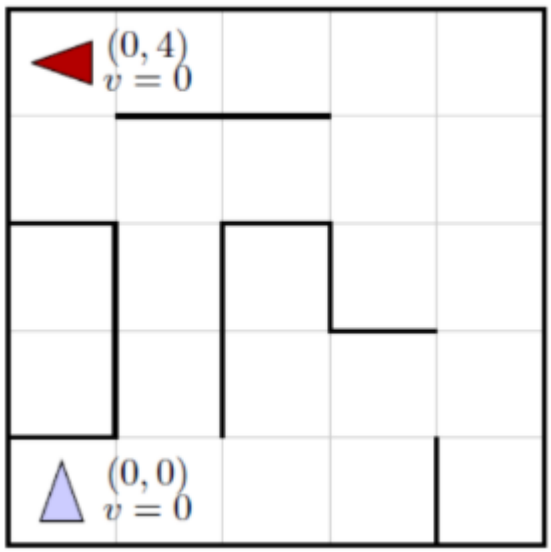
Để A* Graph Search bảo đảm tối ưu theo cách cài đặt chuẩn không reopen node, heuristic cần consistent:

$$h(n) \leq c(n, n') + h(n')$$

Consistency làm cho giá trị $f(n)$ không giảm dọc theo một đường đi, nhờ đó khi một node được lấy ra với chi phí tốt nhất thì không cần mở lại.

CHƯƠNG 2: TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1. Tìm kiếm và heuristic



a. Chuỗi hành động ngắn nhất theo nhánh đi qua $(1, 2)$:

R, F, M, M, S, L, F, S, R, F, S, L, F, M, S, L, F, M, M, S, L, F, S, L, L, F, S, R, F, M, S, L, F, S, L, F, M, M, S.

Sau chuỗi trên, xe đạt $(0, 4)$, hướng Tây và $v = 0$.

b. Mỗi trạng thái có dạng $s = (x, y, d, v)$. Có MN vị trí, 4 hướng và $V_{max} + 1$ giá trị vận tốc. Vì vậy: $|S| = 4MN(V_{max} + 1)$

c. **Có**. Ví dụ $s = (3, 0, \text{Đông}, 2)$. Phía trước là tường nên Faster, Maintain và Slower đều làm xe va tường; xe cũng không thể rẽ vì $v \neq 0$. Vì vậy trạng thái này không có nút con.

d. Hệ số phân nhánh lớn nhất là $b_{max} = 3$. Ví dụ tại $s = (2, 0, Bắc, 0)$, ba hành động hợp lệ là Faster, rẽ trái và rẽ phải.
e. **Không.** Một hành động có thể di chuyển nhiều hơn một ô khi $v > 1$, nên Manhattan distance có thể đánh giá cao chi phí theo số hành động. Ví dụ $n = (8, 4, Tây, 0)$, goal $= (0, 4, Tây, 0)$: $h(n) = 8$, nhưng chỉ cần 6 hành động, nên $h(n) > h^*(n)$.
f. **Có.** Trong một hành động xe di chuyển tối đa V_{max} ô. Vì vậy $h(n) = \frac{D_M(n)}{V_{max}} \leq h^*(n)$ Các ràng buộc về tường, vận tốc và hướng chỉ làm chi phí thực tế tăng lên, nên heuristic này không đánh giá cao chi phí tối ưu.
g. **Không.** Heuristic không admissible có thể thay đổi thứ tự mở rộng node nhưng không làm mất tính đầy đủ trong điều kiện bài toán: đồ thị hữu hạn, chi phí mỗi hành động dương và heuristic không âm. Nếu tồn tại lời giải, A* Tree Search vẫn tìm được lời giải.
h. **Có.** Heuristic không admissible có thể overestimate chi phí còn lại, làm A* ưu tiên một đường đi không tối ưu và trả về goal có chi phí lớn hơn đường tối ưu. Vì vậy optimality có thể thay đổi.
i. Lý do hợp lý để sử dụng heuristic không admissible
    1. Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.
    2. Heuristic không admissible có thể gần chi phí thực hơn, dù đôi khi đánh giá cao, nhờ đó có thể mở ít node hơn.
    3. Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.
    4. Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

a. Breadth-First Search (BFS)

Bất đẳng thức:

$$d(n) < d(n_s)$$

b. Uniform-Cost Search (UCS)

Bất đẳng thức:

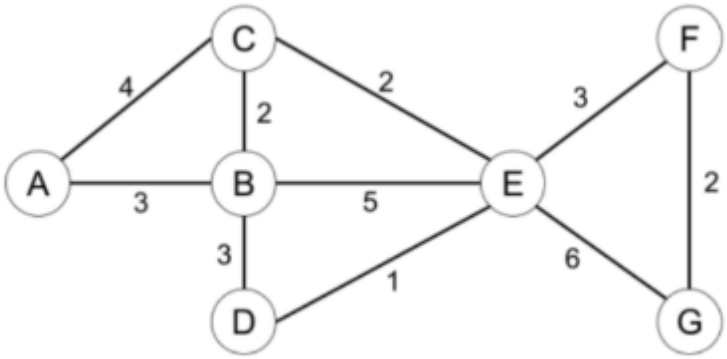
$$g(n) < g(n_c)$$

c. A* Search, giả sử h là heuristic nhất quán

Bất đẳng thức:

$$g(n) + h(n) < g(n_c)$$

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4

E	3	5
F	2	1
G	0	0

a. Bảng đáp án câu

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS		**X**		
Greedy với h_1				**X**
Greedy với h_2	**X**			
*A với h_1**			**X**	
*A với h_2**		**X**		

b. Chi phí của đường đi tối ưu từ A đến G theo UCS

Đường tối ưu:

$$A \rightarrow C \rightarrow E \rightarrow F \rightarrow G$$

Chi phí:

$$4 + 2 + 3 + 2 = 11$$

Đáp án:

11

Đây là đường tối ưu duy nhất trong các đường chính của đồ thị.

c. Heuristic h_1 có admissible không? Có consistent không?

Admissible: ☐ Có ☑ Không

Consistent: ☐ Có ☑ Không

d. Heuristic h_2 có admissible không? Có consistent không?

Admissible: ☑ Có ☐ Không

Consistent: ☐ Có ☑ Không

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a. Đúng/Sai

i. Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

Sai. Tree Search không lưu explored set nên DFS có thể đi vô hạn trong một chu trình, dù số trạng thái của đồ thị là hữu hạn.

ii. Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

Đúng. BFS duyệt theo từng mức độ sâu. Với branching factor hữu hạn, nếu goal tồn tại ở độ sâu hữu hạn thì BFS chắc chắn sẽ tới được goal.

iii. Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

Đúng. Iterative Deepening tăng dần giới hạn độ sâu. Nếu goal ở độ sâu hữu hạn d, thuật toán sẽ tìm thấy khi depth limit đạt d.

iv. Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

Sai. Graph Search loại bỏ các trạng thái trùng lặp nên frontier có thể nhỏ hơn hoặc bằng Tree Search. Nếu đồ thị là một cây, hai frontier có thể bằng nhau.

v. Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

Sai. Iterative Deepening Tree Search có thời gian tương tự BFS và bộ nhớ tương tự DFS. Nhưng Graph Search phải lưu explored set, nên bộ nhớ có thể lớn hơn DFS.

vi. Nếu $h_1(s)$ và $h_2(s)$ đều là heuristic consistent thì $\min(h_1(s), h_2(s))$ cũng nhất thiết consistent.

Đúng. Nếu $h(n) = \min(h_1(n), h_2(n))$, chọn heuristic đạt giá trị nhỏ nhất tại n'. Vì từng heuristic đều consistent nên $h(n) \leq c(n, n') + h(n')$. Do đó $\min(h_1, h_2)$ vẫn consistent.

b. Thiết kế heuristic admissible nhưng không consistent

Trạng thái	**h(s)**
S	**4**
A	**0**
C	**2**
G	**0**

Heuristic admissible vì:

$$h(S) = 4 \leq 4, h(A) = 0 \leq 3, h(C) = 2 \leq 2, h(G) = 0$$

Nhưng không consistent vì trên cạnh $S - A$:

$$h(S) = 4 > c(S, A) + h(A) = 1 + 0 = 1$$

Do đó đây là một heuristic admissible nhưng không consistent, đúng yêu cầu phần b của đề.